Project Overview

What You're Building

A fitness app that generates personalized workouts using AI and minimal user input, reducing decision fatigue at the gym.

Why You're Building It

To learn how to ship a real, end-to-end product using AI as a development partner—covering architecture, tooling, deployment, and iteration—by building real things, not simulating process.

This is not a course. This is a build process.

Success Criteria

Product Success

1. Functional web app that generates workouts
2. Deployed and shareable
3. Used by real people
4. Produces varied, appropriate results from minimal input

Learning Success

1. Comfortable guiding AI to generate and modify code
2. Understand how frontend, backend, APIs, and data fit together
3. Can reason about architecture and tradeoffs
4. Confident shipping independently, not just experimenting
5. Speak effectively with developers using shared mental models and terminology

Core Principles

Ship early, iterate often

Working software beats complete plans.

Build vertical slices

Each step should result in something usable, even if incomplete.

AI does the labor, you make decisions

You are responsible for judgment, not syntax.

Design emerges through use

Polish follows proof, not the other way around.

Every step should expose a real constraint

Confusion and friction are signals, not failures.

Tech Stack

Default stack. Change only if blocked.

Frontend

1. Framework: React
2. Styling: Tailwind CSS
3. UI Generation: Lovable (for rapid prototyping and reference)

Backend

1. Runtime: Node.js
2. Framework: Express (or the simplest backend that keeps secrets secure)
3. AI Integration: Claude API (Anthropic)

Database

1. Supabase (PostgreSQL) or Firebase
2. Stores user data, workout history, preferences

Deployment

1. Frontend: Vercel
2. Backend: Railway or Render
3. Mobile (optional): Capacitor

Development Tools

1. Claude Code (generation, refactoring, debugging)
2. Claude chat (architecture, reasoning, explanation)
3. Git + GitHub
4. VS Code
5. Task tracking tools introduced only when complexity demands it

Step 0 — Tooling & Environment Setup

Goal

Be able to build, deploy, and iterate without friction.

What This Step Exposes

1. How developers reduce setup friction
2. How tooling enables speed later
3. Where automation replaces manual effort

Key Actions

1. Install Node.js, Git, and a code editor
2. Configure Git identity
3. Set up Claude Code and API access
4. Initialize a Git repository
5. Commit and deploy something small

Definition of Done

1. You can go from idea → code → deployed URL without blockers
2. You understand where friction appears in the toolchain

Step 1 — Define the Smallest Shippable Product

Goal
Decide what must exist for the app to be useful—nothing more.
Questions to Answer

1. What input does the user provide?
2. What output do they receive?
3. If everything else disappeared, would this still be valuable?

Key Actions

1. Write a one-paragraph value statement
2. Define a single primary user flow
3. Limit required inputs to three or fewer
4. List explicitly deferred ideas
5. Draft an initial AI workout prompt and test it in isolation

This is where teams eventually introduce task tracking tools—not to plan better, but to avoid losing context once work no longer fits in one person's head.
Definition of Done

1. You can explain the app in 30 seconds
2. There is exactly one core flow
3. You know what you are not building yet

Step 2 — First Vertical Slice (UI + Output)
Goal
Make the app feel real as fast as possible.
What This Step Exposes

1. How developers trade correctness for momentum
2. How abstractions are replaced later
3. How design decisions surface through use

Key Actions

1. Scaffold a React app
2. Build a minimal input form
3. Display workout output (fake or real—whichever is faster)
4. Apply just enough styling to be usable
5. Deploy early

Fake data is acceptable here. The point is to validate flow, not logic.
Definition of Done

1. The app is deployed
2. Someone else can click through it

3. The core flow exists end-to-end

Step 3 — Output Quality & Reliability
Goal
Replace placeholders with real value and make it dependable.
What This Step Exposes

1. API boundaries
2. Async behavior
3. Error states as UX, not edge cases

Key Actions

1. Integrate the Claude API
2. Handle loading, error, and empty states
3. Iterate on prompt quality
4. Add constraints to keep outputs usable

If momentum is high, this step often collapses into Step 2. The separation exists to teach replacement of abstractions, not to enforce sequence.
Definition of Done

1. Real workouts are generated consistently
2. Outputs are usable and predictable
3. Failure cases are handled intentionally

Step 4 — Persistence & Core Infrastructure
Goal
Turn a demo into something that remembers.
What This Step Exposes

1. Why backends exist
2. Why secrets don't live in the frontend
3. How data shapes product decisions

Key Actions

1. Introduce a backend or serverless layer
2. Move AI calls server-side
3. Add basic persistence (e.g., workout history)
4. Keep authentication minimal

Architecture will change here. That's expected.
Definition of Done

1. Data persists across sessions

2. Frontend and backend communicate cleanly
3. Secrets are no longer exposed

Step 5 — Remove Friction (Only What Blocks Use)

Goal

Make the product learnable and shareable.

What This Step Exposes

1. How developers decide what not to fix
2. The difference between polish and clarity

Key Actions

1. Fix confusing flows
2. Improve feedback and copy
3. Resolve obvious bugs
4. Ignore performance and animation work unless users complain

Definition of Done

1. You're comfortable sharing the link
2. Users can complete the core flow without explanation

Step 6 — Real-World Feedback Loop

Goal

Learn what actually matters outside your head.

Key Actions

1. Share with a small group
2. Observe confusion and failure points
3. Identify patterns
4. Fix only what blocks usefulness

Definition of Done

1. The core value holds up with real users
2. Evidence guides next decisions

Step 7 — Mobile Packaging (Optional)

Goal

Extend reach, not rebuild the product.

Only do this after the web experience works.

Step 8 — App Store Submission

Goal

Complete the lifecycle and experience shipping to production.

Continuous Practices

Version Control

1. Commit small and often
2. Use history as a safety net, not a trophy

AI Usage

1. Ask why code works, not just what to paste
2. Treat AI as a junior engineer you must review

Decision Logging

1. Record decisions and reversals
2. Learning happens in the deltas

Progress Over Timeline
Focus on outcomes, not speed.
Each step is complete when its definition of done is met—not when a clock says so. Some steps take minutes. Others surface weeks of learning.
That's normal.
What This Is — and Isn't
This Is

1. A learning-through-shipping framework
2. A way to think like a developer while building
3. A reference you return to, not a checklist you finish

This Is Not

1. A course
2. A performance metric
3. A rigid process

Post-Launch
Shipping is not the end. It's where learning compounds.